

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Diseño de Software

Tercer Proyecto: Servidor nivel producción en Azure para un juego por turnos

ASHENFALL, servidor autoritativo de un duelo 1v1 por turnos

Estudiante:

Kevin Miranda Méndez

Carné 2024120764

Repositorio:
https://github.com/kevinmirandaa/ashenfall-game

Servidor en Azure:
https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io

Profesor:

Jonathan Solis Parajeles

II Semestre 2026

1. Introducción

Este documento describe el servidor que construí para el tercer proyecto del curso: un servidor de nivel producción, desplegado en Azure, para un juego multijugador por turnos. Como indica el enunciado, se evalúa el servidor y no el juego, por lo que aquí se explica qué servidor se hizo, dónde corre, con qué tecnologías, cómo está configurado, cómo se protege, cómo se despliega y cómo se midió. La sección 10 es la lista de todo lo que pide el enunciado, con su estado.

El servidor se llama ASHENFALL y atiende duelos 1v1 por turnos de proyectiles. Es autoritativo: el cliente solo envía intenciones, y el servidor valida cada jugada y es la única fuente de verdad. Cada partida es un registro de eventos en PostgreSQL que se puede reproducir jugada por jugada y que sobrevive a reinicios y publicaciones.

Qué	Dónde
Servidor en Azure	https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io
API REST (OpenAPI 3.1)	https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io/docs
Repositorio en GitHub	https://github.com/kevinmirandaa/ashenfall-game
Versión evaluada	Etiqueta v1.0.0
Tablero de trabajo (Jira)	Proyecto SCRUM
Documentación en el repositorio	Carpeta docs/ (índice en docs/README.md)

2. El servidor y el juego

Aspecto	Detalle
Lenguaje	TypeScript estricto (sin any) sobre Node.js 22
Servidor	Fastify 5; WebSocket con la librería ws
Datos	PostgreSQL 16 (eventos y datos) y Redis 7 (difusión, presencia y límites)
Dominio	Módulo puro en packages/domain, sin red, base de datos ni framework
Contratos	OpenAPI 3.1 en /docs y AsyncAPI 3.0
Dónde corre	Azure Container Apps, región canadacentral, grupo rg-ashenfall-prod
Infraestructura	Bicep y GitHub Actions con OIDC, sin secretos
Cliente de prueba	Phaser 3, servido por el propio servidor

El juego es el pretexto: un duelo 1v1 por turnos de proyectiles. Cada jugador empieza con 100 puntos de vida, el turno dura 30 segundos y la partida tiene como máximo 12 rondas. Se dispara arrastrando hacia atrás y soltando, como una tirachinas. Hay siete arenas, y el servidor sortea el terreno y los obstáculos de cada partida. Las reglas son propias, y el dominio decide sin ambigüedad si una jugada es válida y cuándo termina la partida. La posición del rival es pública por diseño, pero hay vista por jugador: el rival nunca recibe el ángulo, la potencia ni el identificador de la jugada.

3. Arquitectura

El motor de reglas es un módulo de dominio puro: el servidor conoce al dominio y el dominio no conoce a nadie, y ESLint lo hace cumplir. La topología es la de referencia del enunciado. Redis corre como una Container App interna en vez de Azure Cache for Redis, porque el costo de este no cabe en el crédito de Azure for Students.

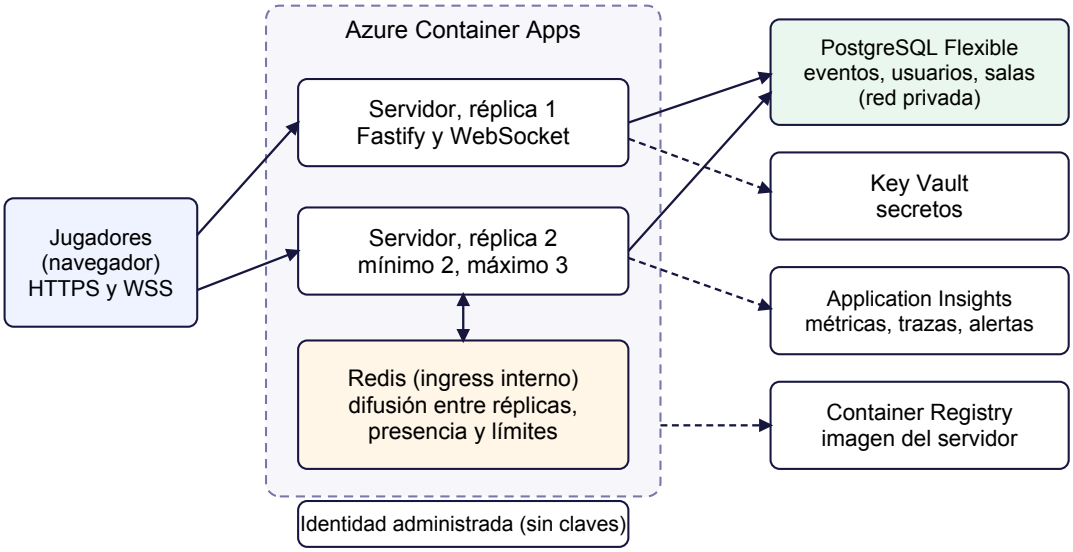


Figura 1. Topología de producción en Azure.

3.1 Patrones y decisiones

- Event sourcing: cada partida es un log de eventos con número de secuencia (seq); el estado es el resultado de reproducirlo, así cualquier réplica puede atenderla.
- Orden por partida: fila de comandos por partida en cada réplica y clave primaria (match_id, seq) entre réplicas; el conflicto se resuelve releyendo el log y decidiendo de nuevo.
- Bus de eventos sobre Redis: cada réplica entrega a sus conexiones los eventos ya confirmados; si se pierde uno, el cliente detecta el hueco en seq y pide el estado con sync.

- Vista por jugador: cada evento se proyecta a lo que ese jugador puede ver.
- Plazos en PostgreSQL: todas las réplicas barren los plazos vencidos, sin réplica líder.

3.2 Flujo de una jugada

El cliente envía shoot con un actionId, la versión esperada, el arma, el ángulo y la potencia. La pasarela valida el esquema, el tamaño y la tasa. El motor encola la jugada, reproduce el estado desde PostgreSQL y el dominio decide: turno, versión y actionId. Si se aplica, se guardan los eventos con (match_id, seq) y se publican por Redis; cada réplica los entrega con la vista de cada jugador y el tirador recibe su ack.

4. Seguridad

Tema	Cómo se resuelve
Tokens	JWT HS256 de 15 minutos, con emisor y audiencia verificados; refresh token rotativo de 30 días, solo con su hash guardado; reutilizar uno revoca la familia.
Contraseñas	Argon2id (19 MiB, 2 iteraciones, paralelismo 1), acotado a una a la vez con fila de 32; si se llena, responde 503.
WebSocket	Ticket de un solo uso de 30 s en el handshake; sin ticket válido, se cierra. Origen restringido (un Origin ajeno recibe 403).
Autorización	Solo los participantes actúan en su partida (lo ajeno responde 404), solo juega quien tiene el turno y solo el administrador usa /v1/admin, con auditoría.
Validación	Cada mensaje se valida contra su esquema (tipos, rangos y 4 KiB) antes del dominio; uno inválido no cierra la partida.
Límites	600 peticiones por minuto por usuario o IP, 20 por minuto en autenticación, 10 logins por minuto por cuenta; HTTP 429 con Retry-After; 20 mensajes por segundo por conexión; 3 conexiones por usuario.
Transporte	Solo HTTPS y WSS (http redirige con 301); HSTS, CSP y nosniff con helmet; CORS vacío en producción (mismo origen).
Secretos	jwt-secret y la cadena de Application Insights están en Key Vault y se leen con identidad administrada; ninguno está en el repositorio (gitleaks revisa el historial).
Identidad y red	PostgreSQL con Microsoft Entra, sin contraseña y sin acceso público; Redis con ingress interno; el pipeline entra con OIDC sin secretos.
Cadena de suministro	Dependabot semanal, CodeQL, gitleaks y Trivy (falla ante vulnerabilidades críticas) en CI; imagen con usuario no root.
Información oculta	El rival nunca recibe ángulo, potencia ni actionId, ni por WebSocket, ni por REST, ni en logs; hay pruebas en cada canal.

Verificado en Azure el 7 de noviembre de 2026: http responde 301, hay HSTS y CSP, un Origin ajeno recibe 403, el login devuelve 429 con Retry-After, los secretos son referencias a Key Vault, PostgreSQL no tiene acceso público y el environment prod exige un revisor. El modelo de amenazas completo está en docs/threat-model.md y el detalle de seguridad en docs/seguridad.md.

5. Sesiones, concurrencia y contratos

- Jugadas simultáneas: una sola se confirma por (match_id, seq); la otra se relee y se decide contra el estado nuevo.
- Duplicados e idempotencia: el reenvío de un actionId devuelve un ack marcado como duplicado y no se aplica dos veces.
- Estado viejo: la versión esperada distinta de la actual se rechaza con STALE_VERSION.
- Reconexión: heartbeat cada 15 s, snapshot al reconectar y gracia de 60 s configurable; al vencer, abandono. Dos turnos agotados seguidos también cuentan como abandono.
- Durabilidad y aislamiento: nada confirmado vive solo en memoria y un fallo en una partida no afecta a las demás ni tumba el proceso.
- Limpieza: las partidas inactivas terminan por sus plazos, las salas sin iniciar se cierran a la hora y la cola expira.
- Contratos: 30 rutas REST en OpenAPI 3.1 (/docs) con errores RFC 9457, y 7 mensajes WebSocket versionados en AsyncAPI 3.0; ambos se generan desde el código y una prueba falla si dejan de coincidir.
- Códigos de cierre: 1001 cierre ordenado, 4003 usuario suspendido, 4008 demasiadas conexiones, 4009 heartbeat perdido y 4029 límite de tasa.

6. Rendimiento y optimización

El bot de apps/bot juega partidas reales por HTTP y WebSocket y mide el tiempo hasta que ambos jugadores reciben el resultado. Las metas del enunciado son p95 menor a 200 ms, ninguna jugada perdida o duplicada y error menor a 1 %. En una réplica de 1 CPU y 1 GB, a un disparo por segundo y partida, se cumplen las tres metas:

Partidas	Jugadas por s	p95	CPU réplica	Memoria
100	92	14 ms	13 %	56 MB
400	369	5 ms	21 %	66 MB
800	737	7 ms	33 %	75 MB

El consumo es de unos 0,03 puntos de CPU y 0,02 MB por partida activa. El punto de quiebre está cerca de 1 800 jugadas por segundo por réplica (a 400 partidas con diez disparos por segundo, el p95 llega a 198 ms). Los mensajes son compactos: JSON de unos 302 bytes de media.

Optimización	Antes	Después
p95 con 100 clientes enviando logins fallidos (Argon2id acotado)	202 ms	109 ms
Preparar 100 partidas con 200 cuentas (parámetros mínimos de OWASP)	57 s	5 s
Idas y vueltas a PostgreSQL por jugada	7	5
Jugadas por segundo en saturación	1 593 a 1 647	1 754 a 1 839
Imagen Docker	565 MB	387 MB

En Azure (7 de noviembre de 2026), con partidas reales contra la URL pública, tres corridas dieron 1 279 jugadas confirmadas y 46 partidas contrastadas con su replay del servidor, con 0 jugadas perdidas, 0 duplicadas y 0 errores. Una de ellas cruzó la publicación de una revisión nueva con 6 partidas abiertas, y las jugadas se repartieron entre las réplicas. La latencia medida por el propio servidor tuvo media de unos 17 ms; de extremo a extremo desde Costa Rica, el p95 fue de 216 ms por el viaje por internet. Los comandos para reproducir todo están en docs/load-report.md.

7. Azure, infraestructura y CI/CD

Recurso	Configuración
ashenfall-server-prod (Container App)	0,5 vCPU y 1 GiB; 2 a 3 réplicas; escala por 50 solicitudes concurrentes; sondas de vida y disponibilidad; 120 s de gracia al apagar; ingress HTTPS externo
ashenfall-redis-prod (Container App)	Redis 7, 0,25 vCPU y 0,5 GiB, ingress interno
psql-ashenfall-prod-3fiowy	PostgreSQL 16 Flexible, B1ms, 32 GB, red privada, solo Entra
kv-ash-prod-3fiowy	Key Vault con RBAC: jwt-secret y cadena de Application Insights
acrashenfallprod3fiowy	Container Registry Basic; imagen etiquetada con el sha del commit
id-ashenfall-prod	Identidad administrada con AcrPull, Key Vault Secrets User y administración de Entra en PostgreSQL
vnet-ashenfall-prod	Red virtual con subredes para Container Apps y PostgreSQL, y DNS privado
log-ashenfall-prod y appi-ashenfall-prod	Log Analytics (30 días) y Application Insights

- Infraestructura como código: todos los recursos están en Bicep (infra/main.bicep) y un ambiente nuevo se levanta con una sola ejecución del pipeline; el 7 de noviembre de 2026 el pipeline creó dev y prod desde cero.
- CI: en cada pull request, build, tipos, linter, 294 pruebas con PostgreSQL y Redis reales, validación de contratos, CodeQL, gitleaks y Trivy.
- Despliegue: entra a Azure con OIDC; construye y publica la imagen, crea jwt-secret en Key Vault si no existe y despliega. Dev es automático y prod espera la aprobación manual de un revisor.
- Publicaciones sin cortar partidas: la revisión nueva nace al 0 % del tráfico, se prueba por su propia URL y recibe el 100 %; la anterior queda como destino de rollback y las viejas se desactivan, cerrando sus conexiones de forma ordenada (código 1001).
- Rollback: scripts/rollback.sh pasa el tráfico a la revisión anterior; se probó en producción y tardó 22 segundos.
- Observabilidad: OpenTelemetry hacia Application Insights con las métricas ashenfall.matches.active, ashenfall.connections, ashenfall.shots y ashenfall.shot.latency; trazas con el id de partida y de jugador; logs estructurados; sondas /health/live y /health/ready; tres alertas (reinicios, CPU de PostgreSQL y latencia).

Límite de la suscripción: Azure for Students permite un solo entorno de Container Apps por suscripción, así que dev y prod no pueden existir a la vez. Dev se desplegó y probó con el pipeline y se borró para dejar sitio a prod; el despliegue automático a dev queda apagado.

8. Pruebas, costo y cómo correrlo

Hay 294 pruebas en 24 archivos (dominio, API, WebSocket, concurrencia, reinicios, seguridad y resiliencia), en verde en CI. La estimación de costo de prod es de unos 47 a 115 USD al mes (Container Apps, PostgreSQL, Redis, registro y logs), con precios de lista y sin descontar cuotas gratuitas; el detalle está en docs/azure-cost.md. Los recursos de dev se apagan o escalan a cero cuando no se usan.

Para correr el servidor en local, en menos de 10 minutos, solo hace falta Docker Desktop: git clone https://github.com/kevinmirandaa/ashenfall-game.git, luego cd ashenfall-game y docker compose up --build. Queda en http://localhost:8080, con la API en /docs. Para correr las pruebas: npm ci, docker compose up -d --wait postgres redis y npm test.

9. Limitaciones

- Dev automático: por el límite de un entorno de Container Apps por suscripción, dev y prod no pueden coexistir.
- Rendimiento en Azure: se midió desde internet y con pocas partidas; el punto de quiebre solo está medido en local.
- Alertas: existen y están activas, pero no se ha visto que disparen y no envían correo.
- Navegadores: el cliente de prueba se probó en Chromium y en emulación móvil; Safari y Firefox no se probaron con navegadores reales.

10. Cumplimiento del enunciado

Lista de todo lo que pide el enunciado y su estado. Más detalle, con 94 puntos y su evidencia, en docs/cumplimiento-enunciado.md.

Requisito del enunciado	Cómo se cumple	Estado
El juego		
Por turnos	El estado guarda quién juega; turno de 30 s	Completado
Multijugador (2 o más humanos)	Cola rápida y salas privadas de 2	Completado
Original	Reglas propias de proyectiles	Completado
Reglas verificables	El dominio valida cada jugada y el fin	Completado
Información oculta (recomendada)	Vista por jugador; posición pública por diseño	Completado
Requisitos funcionales		
Usuarios	Registro, login, logout, refresh rotativo y perfil	Completado
Salas	Públicas y privadas con código; unirse, salir y listo	Completado
Emparejamiento	Cola rápida con espera máxima configurable	Completado
Turnos	Solo el del turno; error tipado; temporizador	Completado
Conexión	Heartbeat, snapshot y gracia configurable	Completado

Requisito del enunciado	Cómo se cumple	Estado
Fin de partida	Resultado inmutable y replay jugada por jugada	Completado
Historial	Paginado; solo ve sus partidas	Completado
Administración	Solo administrador y con auditoría	Completado
Reglas del servidor		
Servidor autoritativo	El cliente envía intenciones	Completado
Orden por partida	Fila por partida y clave (match_id, seq)	Completado
Idempotencia	actionId; ack duplicado	Completado
Vista por jugador	Sin filtración en mensajes, REST ni logs	Completado
Durabilidad	Estado en PostgreSQL; sobrevive a reinicios	Completado
Aislamiento	Un fallo no afecta a otras partidas	Completado
Limpieza	Plazos y expiración automática	Completado
Reloj y azar del servidor	Generador criptográfico del servidor	Completado
Lenguaje, comunicación y contratos		
Lenguaje tipado estricto	TypeScript strict, sin any	Completado
Dominio puro	packages/domain sin red ni base de datos	Completado
WSS sin polling del turno	ws sobre WSS; eventos empujados	Completado
Protocolo con AsyncAPI 3.0	7 mensajes versionados	Completado
OpenAPI 3.1 en /docs	30 rutas; Swagger UI	Completado
Errores RFC 9457	Problem Details en toda la API	Completado
Validación de mensajes	Esquema y 4 KiB antes del dominio	Completado
Heartbeat y códigos de cierre	Ping y pong; 1001, 4003, 4008, 4009, 4029	Completado
Seguridad		
Autenticación	JWT, Argon2id y ticket de WebSocket	Completado
Autorización	Participantes, turno y rol administrador	Completado

Requisito del enunciado	Cómo se cumple	Estado
Límites	429 con Retry-After; mensajes y conexiones	Completado
Transporte	HTTPS y WSS, CORS, cabeceras de seguridad	Completado
Secretos	Key Vault con identidad administrada	Completado
Cadena de suministro	Dependabot, CodeQL, gitleaks, Trivy; no root	Completado
Rendimiento		
Prueba de carga y metas	p95 de 15 ms o menos; 0 perdidas; 0 errores	Completado
Capacidad y punto de quiebre	CPU y memoria por partida; quiebre medido	Completado
Mensajes compactos	JSON de unos 302 bytes	Completado
Optimización medida	Dos, con cifras antes y después	Completado
Azure y entrega continua		
Infraestructura como código	Bicep; ambiente desde cero con el pipeline	Completado
CI/CD con dev automático y prod manual	Prod con revisor; dev automático con limitación (nota)	Completado *
Publicaciones sin cortar partidas	Tráfico al 0 %, drenado, cierre 1001	Completado
Rollback en menos de 5 minutos	Probado en prod: 22 s	Completado
Identidad administrada	Sin cadenas de conexión ni claves	Completado
Observabilidad	OpenTelemetry, métricas, sondas y 3 alertas	Completado
Costo	Estimación mensual; dev a cero	Completado
Entregables		
Repositorio con todo lo pedido	Código, IaC, pipelines, contratos, pruebas, bots y cliente	Completado
README	Local en menos de 10 minutos, pruebas y URL	Completado
URL pública en Azure	Disponible durante la revisión	Completado
Versión etiquetada v1.0.0	Etiqueta publicada	Completado
Documentación	Este documento y docs/	Completado

Total: 51 requisitos, todos completados. * Azure for Students permite un solo entorno de Container Apps por suscripción, así que dev y prod no pueden coexistir: dev se desplegó y probó con el pipeline y se borró, y el despliegue automático a dev queda apagado; prod tiene aprobación manual.

11. Conclusión

El servidor de ASHENFALL cumple lo que pide el proyecto: es autoritativo, seguro y observable, atiende muchas partidas simultáneas con orden e idempotencia, funciona con varias réplicas, se publica sin cortar partidas y se revierte en segundos, y está desplegado en Azure con infraestructura como código, identidad administrada y aprobación manual a producción. Las cifras salen de ejecuciones reales y reproducibles con los bots del repositorio.